Data Science Methods for Cognitive Science

Dan Hicks, hicks.daniel.j@gmail.com

2020-08-17

Contents

Preface **5**

Week 0: Setting up **7**
0.1 R and RStudio: The Mac Way 7
0.2 R and RStudio: The Cloud Way 8
0.3 Slack 8
0.4 GitHub 9
0.5 Screen Layouts 9
0.6 Pre-Class Survey 9
0.7 A Note on Accessibility 10

1 Introduction **11**
1.1 Land Acknowledgement 11
1.2 About me 11
1.3 What this course isn't, and is 12
1.4 Learning outcomes 12
1.5 Notes on some course design decisions 13

I Some fundamental tools **15**

2 R Basics **17**
2.1 Let's get started with RStudio 17
2.2 Base R data types 20
2.3 *[examining variables]* 22
2.4 *[Missing data]* 22

3 Git and Version Control **23**

4 Functional Programming and Data Science **25**

5 Warnings, Errors, and Getting Help **27**
5.1 Recommended reading and watching 27
5.2 Messages, warnings, and errors 27

5.3 Where to go for help 27
5.4 Writing a reproducible example: `reprex` 29

II Exploratory Data Analysis 31

6 Waterfalls, Spirals, and Philosophy of Science 33

7 Exploratory Data Analysis and the Ladder of Models 35

III Cleaning Data 37

IV Reproducibility 39

V Case Study: The COMPAS Dataset 41

Preface

This document is a set of notes (and perhaps later a textbook) for a course on data science methods for graduate students in social and behavioral sciences, taught by Professor Dan Hicks (they/them), UC Merced.

Week 0: Setting up

Before the first day of class, you'll need to set up a few things: **R, RStudio, Slack, Zoom, and a GitHub account**. I'd also like you to fill out a **pre-class survey**. R is the primary piece of software we'll be learning to use. RStudio is an IDE (integrated development environment) for using R. (Think of R as the engine of a car and RStudio as the dashboard, steering wheel, and pedals.) Slack will be the main way we communicate; we'll use Zoom for class time. GitHub is a service for sharing and collaborating on software engineering projects.

Table 1: Key links for setup

R	https://cran.r-project.org/
RStudio	https://rstudio.com/products/rstudio/download/
Slack	http://datascienceme-ke13588.slack.com/
GitHub	https://github.com/; https://github.com/data-science-methods
Survey	https://ucmerced.az1.qualtrics.com/jfe/form/SV_eVU8JWREUu4UNH7

I'll assume you have R and RStudio installed using either *the cloud way* or *the Mac way*. If your computer is running Windows, Linux, or some other operating system that isn't Mac OS, I'm going to assume you have things set up the cloud way. These other operating systems are perfectly fine. But I don't use them, which makes it much harder for me to help you if things aren't working. And Windows handles things like file paths and text file formats in idiosyncratic ways that can cause weird errors.

In either case, all of the software and services we'll be using are either free or freemium. You won't need any of the paid extras for this course.

0.1 R and RStudio: The Mac Way

You can find some notes on accessibility and RStudio here: https://support.rstudio.com/hc/en-us/articles/360044226673-RStudio-Accessibility-Features.

With this way, R and RStudio are installed and run on your computer, and

all of the data files that you work with will also be on your computer. Unless you're running a Chromebook or similar, your computer is almost certainly more powerful than the default instance that RStudio Cloud gives you (see below), and you can use familiar tools to do things like organize folders and preview files. But sometimes particular details about the way your computer is set up can cause R and RStudio to do weird things, and these problems can be very difficult to diagnose and solve.

1. Go to https://cran.r-project.org/. Download and install the latest release of R for your operating system (which should be Mac OS if you're following this way). As of 2020-06-02, the latest release is 4.0.0.

2. Go to `rstudio.com`. Download and install the latest release of RStudio Desktop for your operating system (which should be Mac OS if you're following this way). As of 2020-06-02, the latest release is 1.3.959.

0.2 R and RStudio: The Cloud Way

(**Note 2017-07-07** Starting in August, RStudio Cloud is moving out of beta and becoming a regular subscription service. Information on pricing is at https://rstudio.cloud/plans/free. We'll have to try it out, but it seems likely that the Free tier won't give you enough compute time ("project hours") for this course. I can subscribe to the Instructor tier, but that has its own limitations. We'll have to see how things go.)

With this way, R and RStudio are installed and run on virtual computers (called *instances*) on a big server somewhere. Each project can have its own instance, meaning that it runs on a nice clean computer and shouldn't have any weirdness. The RStudio Cloud service is currently in beta; it's free to use, but not completely guaranteed to be free of bugs. In order to use the service, you have to be connected to the internet, and you'll have to use the RStudio interface or a terminal window to do things like move files around. Also, the instances aren't very powerful, and so they'll be slow if you try to work with large data sets or do a complex analysis.

1. Go to `rstudio.cloud`. Look for "Sign Up" in the upper right.[1]

0.3 Slack

For Fall 2020, the class Slack channel is at https://datascienceme-ke13588.slack.com. Please use your UC Merced email to sign up.

If you use a screenreader, here are some notes on accessibility and Slack: https://www.marcozehe.de/status-of-the-accessibility-of-slack/.

[1] RStudio gives you the option to log in with Google or GitHub. As a general practice, I recommend always keeping your accounts separate. Use a password manager to generate separate, strong passwords for each site. I use LastPass.

0.4 GitHub

1. Go to `github.com`. You can create a new account right on the front page. I don't care what email address you use here.
2. You can fill in your profile if you want.
3. You'll send me the email address associated with your Github account in the pre-class survey.

0.5 Screen Layouts

In this course, a significant amount of time will be spent on *live coding*. This means that I (or whoever's doing the lecturing) will be writing code in real time, and you'll follow along, transcribing the code into RStudio (or whatever) on your local machine and running it at the same time as me.

Live coding is a great way to learn coding practices. But it requires you to have quick access to 3-5 different things:

- My screenshare (so you can see what I'm typing)
- The course Slack (so you can ask for help or other questions, or make comments)
- Possibly a separate way to put up a flag so that I can see immediately that you have a question
- Your own workspace (RStudio or whatever)
- Possibly a separate app for taking notes

In an ordinary classroom, the first three things would all be handled by the classroom environment. In a fully online setting, everything needs to be handled on your computer.

The most effective way to do this is to have multiple displays. This is also the most expensive.

This blog post discusses alternatives to multiple displays. In particular, the window layout below fits the screenshare, chat, and your workspace on to a single laptop display. You can also have a separate text file open in RStudio for notes. You might consider having Slack open on your phone — so that you can keep an eye on the chat — as well in the background on your computer — so that you can alt-tab to it to actually type things.

0.6 Pre-Class Survey

Please take a few minutes to fill out this survey: https://ucmerced.az1.qualtrics.com/jfe/form/SV_eVU8JWREUu4UNH7.

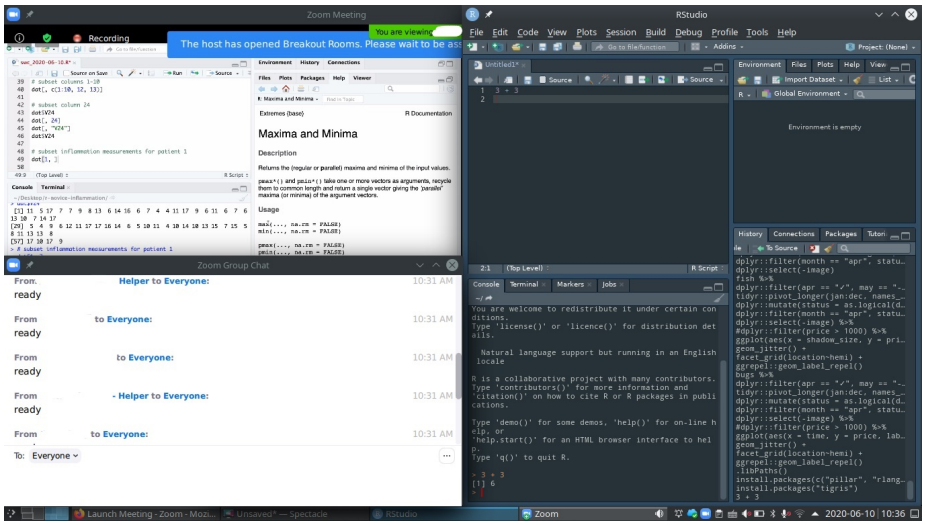


Figure 1: Source: https://carpentries.org/images//blog/2020/06/screen_layouts/02-student-with_participantlist-1366x768.jpg

0.7 A Note on Accessibility

I chose the tools and platforms for this course in part because they're industry-standard. If you pursue a career as a data scientist in industry, you'll be expected to use Slack and Github on a daily basis. And RStudio is by far the most commonly used IDE ("integrated development environment") for R.

However, like many other technologies, they were originally developed using ableist assumptions about "normal" computer users. In response to criticism, the developers of these systems and tools have gone back and made their technologies more accessible. But there may still be barriers to accessibility that I have not anticipated. And of course the pandemic has created additional barriers, such as dependency on screensharing.

If you encounter a barrier to participating in this course — even a small inconvenience — please let me know. Similarly, if you have ideas for making the course more accessible, please share them with me.

Chapter 1

Introduction

1.1 Land Acknowledgement

UC Merced and the City of Merced are on the traditional territory of the Yokut people. This land was stolen by Spanish, Mexican, and American settlers through acts of slavery and genocide. In addition, UC Merced is strongly associated with Ahwahne, known as Yosemite Valley. This valley was the traditional home of the Ahwahnechee people, who were the victims of some especially horrific, state-sponsored genocidal acts. For more on the history of Ahwahne, see https://tinyurl.com/y879jw8s. For more information on land acknowledgments, see https://native-land.ca.

1.2 About me

Dan Hicks is a philosopher turned data scientist turned philosopher.

I use they/them pronouns and identify as nonbinary. I grew up in Placerville, about two hours north of Merced in the Sierra Foothills. One branch of my family came to California during the Gold Rush, so I identify heavily as a Californian and have some complicated feelings about the genocide. I finished my PhD in philosophy of science at Notre Dame in 2012. After that I worked in a series of research positions in academia and the federal government. During 2015-2019 I was using data science methods at least half-time. I joined the faculty at UC Merced in Fall 2019.

- Email: dhicks4@ucmerced.edu
- Student hours: By appointment: https://doodle.com/mm/danhicks/office-hours
- Website: https://dhicks.github.io/

1.3 What this course isn't, and is

Is Not:

- statistics (in the way you think)
- general introduction to software engineering

Is:

- data cleaning, data management, and exploratory data analysis
- habituation to some good software engineering practices that are especially valuable for data science work

1.4 Learning outcomes

By the end of the course, students will be able to

1. apply concepts from software engineering and philosophy of science to methodological decisions in data science, including
    - functional vs. object-oriented programming paradigms [4]
    - waterfall vs. spiral model of development
    - ladder of models
    - reproducibility vs. robustness vs. replicability
    - value and hazards of open data
    - thick data [19]
2. use exploratory data analysis techniques and tools to identify potential data errors and potential phenomena for further analysis
    - checklist [11]
    - `skimr`, `visdat`
    - 1D summaries: histograms, density, ecdf plots
    - 2D summaries: scatterplots, boxplot, violin plot, beeswarm, upset plots; pairs plot (https://www.data-imaginist.com/2019/a-flurry-of-facets/); correlation matrix
3. clean raw data and produce a reproducible codebook for both downstream analysis and public release of data according to FAIR standards [22]
    - [create and use a cloud database to work with big data]
4. manage data, analysis, and outputs for reproducibility [assembling a project compendium; 6] using best practices of data handling, a clear directory structure [23], self-documenting code, version control, build automation, [and continuous integration]
5. apply philosophical and data science concepts to integrated ethical-technical analysis to case studies in algorithmic injustice

- understand the background on the COMPAS case: Angwin et al. [2]; Corbett-Davies et al. [5]
- reproduce several fairness statistics on COMPAS data
- explain tradeoffs between false negatives and false positives in terms of inductive risk
- explain Kleinberg, Mullainathan, and Raghavan [13] impossibility result (not necessarily the proof) and show that it applies to the COMPAS datas
- critically discuss profiling and thresholds [16] as potential explanation for differences in "base rates" of recidivism
- critically locate discussions of fairness and recidivism in a broader social context, and identify implications for working data scientists [17, 10, 8]

1.5 Notes on some course design decisions

R, not Python

- Python is a perfectly good language for general programming
  - And has some advantages over R in certain data science aspects, including working with strings and developing custom data structures
- R is a better language for doing data science [18]
  - Vectors, data frames, and common statistical models are installed and loaded by default
  - R is more functional [24, 14]
- R is also better for beginners
  - CRAN
  - RStudio

Tidyverse R, not base R

- "Base R" refers to the central code for R and the packages included when you install it
  - Base R is universal and free of additional dependencies. Code written entirely in base R will run on any other R installation (with the same version).
  - Because the R Core Development Team has always thought backwards-compatibility is important, base R has some idiosyncracies [12]
- `tidyverse` is a suite of packages developed by the manufacturers of RStudio
  - A very useful set of general tools can be loaded simply by calling `library(tidyverse)`
    - And some more specialized tools are also installed as part of the suite
  - Tidyverse is designed to mitigate a lot of the base R idiosyncracies

  - While making code more human-readable and easier to reason about
  - However, several of its tools use something called "non-standard evaluation" (NSE).
    * NSE makes tidyverse code easier to write for beginners, because you don't have to think as much about where exactly your variables are located
    * At the same time, NSE creates problems for intermediate coders, because passing tidyverse-style "bare names" through function arguments causes cryptic errors

Scripts, not notebooks

- Quite a few scientists like the way notebooks mix together code, visualizations, tables, and text [15]
- Notebooks do not play nicely with version control [1, 20]
- Notebooks encourage bad habits and discourage good ones [7, 9]

Part I

Some fundamental tools

Chapter 2

R Basics

2.1 Let's get started with RStudio

- Quick tour of RStudio's UI
- Let's plot a thing

```r
data(anscombe) ## <https://en.wikipedia.org/wiki/Anscombe%27s_quartet>

anscombe
```

```
##    x1 x2 x3 x4    y1   y2    y3    y4
## 1  10 10 10  8  8.04 9.14  7.46  6.58
## 2   8  8  8  8  6.95 8.14  6.77  5.76
## 3  13 13 13  8  7.58 8.74 12.74  7.71
## 4   9  9  9  8  8.81 8.77  7.11  8.84
## 5  11 11 11  8  8.33 9.26  7.81  8.47
## 6  14 14 14  8  9.96 8.10  8.84  7.04
## 7   6  6  6  8  7.24 6.13  6.08  5.25
## 8   4  4  4 19  4.26 3.10  5.39 12.50
## 9  12 12 12  8 10.84 9.13  8.15  5.56
## 10  7  7  7  8  4.82 7.26  6.42  7.91
## 11  5  5  5  8  5.68 4.74  5.73  6.89
```

```r
str(anscombe)
```

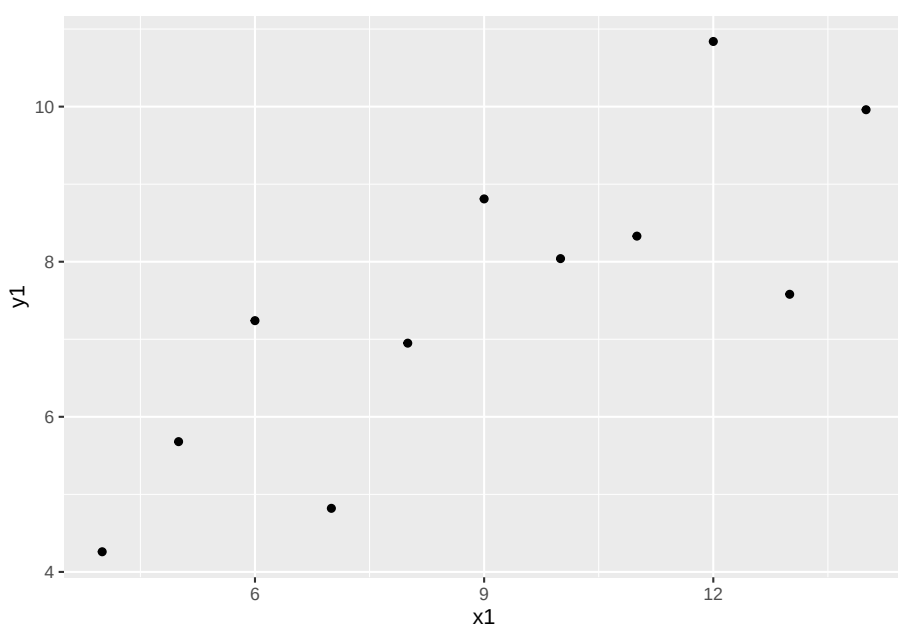
```
## 'data.frame':    11 obs. of  8 variables:
##  $ x1: num  10 8 13 9 11 14 6 4 12 7 ...
##  $ x2: num  10 8 13 9 11 14 6 4 12 7 ...
##  $ x3: num  10 8 13 9 11 14 6 4 12 7 ...
##  $ x4: num  8 8 8 8 8 8 8 19 8 8 ...
##  $ y1: num  8.04 6.95 7.58 8.81 8.33 ...
##  $ y2: num  9.14 8.14 8.74 8.77 9.26 8.1 6.13 3.1 9.13 7.26 ...
```

```
##  $ y3: num  7.46 6.77 12.74 7.11 7.81 ...
##  $ y4: num  6.58 5.76 7.71 8.84 8.47 7.04 5.25 12.5 5.56 7.91 ...
```

```r
summary(anscombe)
```

```
##        x1             x2             x3             x4           y1
##  Min.   : 4.0   Min.   : 4.0   Min.   : 4.0   Min.   : 8   Min.   : 4.260
##  1st Qu.: 6.5   1st Qu.: 6.5   1st Qu.: 6.5   1st Qu.: 8   1st Qu.: 6.315
##  Median : 9.0   Median : 9.0   Median : 9.0   Median : 8   Median : 7.580
##  Mean   : 9.0   Mean   : 9.0   Mean   : 9.0   Mean   : 9   Mean   : 7.501
##  3rd Qu.:11.5   3rd Qu.:11.5   3rd Qu.:11.5   3rd Qu.: 8   3rd Qu.: 8.570
##  Max.   :14.0   Max.   :14.0   Max.   :14.0   Max.   :19   Max.   :10.840
##        y2              y3              y4
##  Min.   :3.100   Min.   : 5.39   Min.   : 5.250
##  1st Qu.:6.695   1st Qu.: 6.25   1st Qu.: 6.170
##  Median :8.140   Median : 7.11   Median : 7.040
##  Mean   :7.501   Mean   : 7.50   Mean   : 7.501
##  3rd Qu.:8.950   3rd Qu.: 7.98   3rd Qu.: 8.190
##  Max.   :9.260   Max.   :12.74   Max.   :12.500
```

```r
library(ggplot2)
ggplot(data = anscombe, aes(x = x1, y = y1)) +
    geom_point()
```



Exercise: plot the other three datasets

```
anscombe$x1
```

```
##  [1] 10  8 13  9 11 14  6  4 12  7  5
```

```
anscombe[['x1']]
```

```
##  [1] 10  8 13  9 11 14  6  4 12  7  5
```

```
anscombe[,'x1']
```

```
##  [1] 10  8 13  9 11 14  6  4 12  7  5
```

```
anscombe[,1]
```

```
##  [1] 10  8 13  9 11 14  6  4 12  7  5
```

```
mean(anscombe$x1)
```

```
## [1] 9
```

```
mean(anscombe$x2)
```

```
## [1] 9
```

```
sd(anscombe$y1)
```

```
## [1] 2.031568
```

```
sd(anscombe$y2)
```

```
## [1] 2.031657
```

```
cor(anscombe$x1, anscombe$y1)
```

```
## [1] 0.8164205
```

Exercise: Calculate correlations for all four x-y pairs

- Some idiosyncracies of my code
  - `'` rather than `"`
  - `##` for documentation comment
  - `=` not `<-` for assignment
- Essential Global Options
  - Never save .Rdata, load workspace
- Super useful custom keyboard commands:
  - Tools → Modify Keyboard Shortcuts…
  - +1, +2, +3

2.2 Base R data types

2.2.1 Numerics

```
foo = 2
bar = 3
foobar = foo + bar
foo / bar
```

```
## [1] 0.6666667
```

```
try(foo / 0)
```

```
## [1] Inf
```

2.2.2 Vectors

```
foo = c(2, 3, 4)
bar = c(5, 6, 7)
foo[2]
```

```
## [1] 3
```

```
foo[2:3]
```

```
## [1] 3 4
```

```
try(foo[1, 3])
```

```
## Error in foo[1, 3] : incorrect number of dimensions
```

```
foo[c(1, 3)]
```

```
## [1] 2 4
```

```
foo + bar
```

```
## [1]  7  9 11
```

```
foo * bar
```

```
## [1] 10 18 28
```

```
## NB difference between vector and length-1 comparisons
bar == foo + 3
```

```
## [1] TRUE TRUE TRUE
```

```
identical(bar, foo + 3)
```

```
## [1] TRUE
```

```r
c(foo, bar)
```

```
## [1] 2 3 4 5 6 7
```

2.2.3 Strings

```r
foo = c('monkey', 'banana')
bar = c('house')
## How to put these together?
try(foo + bar)
```

```
## Error in foo + bar : non-numeric argument to binary operator
```

```r
c(foo, bar)
```

```
## [1] "monkey" "banana" "house"
```

```r
paste(foo, bar)
```

```
## [1] "monkey house" "banana house"
```

```r
paste0(foo, bar)
```

```
## [1] "monkeyhouse" "bananahouse"
```

2.2.4 Factors

```r
foo_str = c('foo', 'foo', 'bar', 'zoo')
as.integer(foo_str)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] NA NA NA NA
```

```r
foo_fct = as.factor(foo_str)
as.integer(foo)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] NA NA
```

```r
levels(foo_fct)
```

```
## [1] "bar" "foo" "zoo"
```

2.2.5 Lists

[todo]

2.2.6 Data frames

[todo]

2.3 *[examining variables]*

- length
- str
- class, typeof
- challenge

2.4 *[Missing data]*

https://gge-ucd.github.io/R-DAVIS/lesson_how_r_thinks_about_data.html#missing_data

Chapter 3

Git and Version Control

- *[motivating example]*
- Version control
  - Basic idea: Tools for tracking changes to code over time
  - Useful for identifying and reversing breaking changes
  - Implementations upload to cloud, track who contributes code
  - Good for collaboration, publishing code
- git
  - One of many version control systems
  - Very popular in part thanks to GitHub, which provides free hosting for open-source projects
- git interfaces
  - command line
  - RStudio projects
  - GUIs: GitHub, Sourcetree
- How to do things with git
  - setup
    - installation
    - user config
  - `git init`
  - `add`
  - `commit`
  - `.gitignore`
  - `fetch` and `pull`
  - `push`
  - forking

  - pull requests
  - browsing history on GitHub
  - stretch goals: `branch`; `checkout`

Chapter 4

Functional Programming and Data Science

- Recommended reading: Chambers [4]; parts II and III of Wickham [21]
- Programming paradigms
  - Procedural: Software is a series of instructions ("procedures"), which the computer carries out in order. Special instructions (if-then, loops) are used to change the order based on inputs or other conditions.
  - Object-oriented: Software is made up of objects, which have properties ("attributes," including other objects) and do things ("methods").
    - Example: A board game consists of 1 or more players, a board, and a deck of cards.
      - Players each have a hand of cards (attribute), can draw cards from the deck into their hand (method, which also changes the deck's attributes), and can play cards to manipulate tokens on the board (method, which also changes the board's attributes).
      - The game itself is an object, with the players, board, and deck for its attributes and methods to things up, run turns in sequence, determine the winner.
      - Separate objects might be responsible for displaying the current state of the game and getting input from the users.
    - Much better than procedural paradigms for *encapsulation*: When a player draws a card from the deck, we just need to know the

inputs and outputs for `Deck.draw()`, not the details of how the deck tracks the order of cards and which ones have already been drawn.
    * Much easier to reason about systems with lots of dynamic, interacting parts.

    * Examples (mostly): Python, C++, Java

  - Functional: Software is made up of functions, which are run sequentially on the inputs

    * Example: Raw data are read in, cleaned, and then a regression model is fit. The model coefficients are tested for statistical significance, and a table and plot are written out to disk.

    * **Data analysis as a building series of pipes**
    * Examples (mostly): R, Lisp, Haskell

  - A couple of extra rules for functional programming

    * *Immutability*: Once the value is assigned to a variable, that value cannot be changed
    * *No side effects*: Functions don't change anything outside of themselves; they only return a value

Chapter 5

Warnings, Errors, and Getting Help

5.1 Recommended reading and watching

- https://stackoverflow.com/a/5963610/3187973
- https://github.com/jennybc/debugging#video

5.2 Messages, warnings, and errors

- *Message*: Things are fine, but here's some information you should know
- *Warning*: Uhhhh I'm gonna keep going, but maybe this isn't what you want
- *Error*: Nope. I'm stopping here. You need to fix the thing.

5.3 Where to go for help

- Isolate the problem
- Restart your session: Session → Restart R
- Local help: `?fun`
- StackOverflow: https://stackoverflow.com/questions/tagged/r
- CRAN → BugReports (usually GitHub Issues)

```r
library(lubridate)
```

```
##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
```

```
##     date, intersect, setdiff, union
```

```
some_data = c('June 2002', 'May 15, 2007', 'August 2007')
mdy(some_data)
```

```
## [1] "2002-06-20" "2007-05-15" "2007-08-20"
```

```
parse_date_time(some_data, orders = 'mdY')
```

```
## Warning: 2 failed to parse.
```

```
## [1] NA                "2007-05-15 UTC" NA
```

Let's try SO: https://stackoverflow.com/search?q=%5BR%5D+lubridate+month-year

```
parse_date_time(some_data, orders = c('mY', 'mdY'))
```

```
## [1] "2002-06-01 UTC" "2007-05-15 UTC" "2007-08-01 UTC"
```

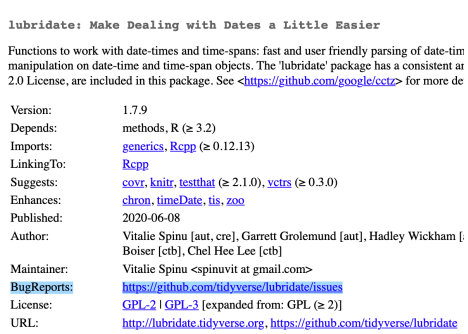
Here's another one:

```
more_data = c('May 7, 2017', 'May 19, 2017', 'May Fifth, 2017')
mdy(more_data)
```

```
## [1] "2017-05-07" "2017-05-19" "2017-05-20"
```

SO doesn't seem so helpful: https://stackoverflow.com/search?q=%5BR%5D+lubridate+written+days

Let's check the CRAN page for `lubridate`: https://cran.r-project.org/web/packages/lubridate/index.html



lubridate: Make Dealing with Dates a Little Easier

Functions to work with date-times and time-spans: fast and user friendly parsing of date-tim
manipulation on date-time and time-span objects. The 'lubridate' package has a consistent ar
2.0 License, are included in this package. See <https://github.com/google/cctz> for more de

Version:	1.7.9
Depends:	methods, R (≥ 3.2)
Imports:	generics, Rcpp (≥ 0.12.13)
LinkingTo:	Rcpp
Suggests:	covr, knitr, testthat (≥ 2.1.0), vctrs (≥ 0.3.0)
Enhances:	chron, timeDate, tis, zoo
Published:	2020-06-08
Author:	Vitalie Spinu [aut, cre], Garrett Grolemund [aut], Hadley Wickham [Boiser [ctb], Chel Hee Lee [ctb]
Maintainer:	Vitalie Spinu <spinuvit at gmail.com>
BugReports:	https://github.com/tidyverse/lubridate/issues
License:	GPL-2 | GPL-3 [expanded from: GPL (≥ 2)]
URL:	http://lubridate.tidyverse.org, https://github.com/tidyverse/lubridate

Figure 5.1: Screenshot of `lubridate` on CRAN, highlighting the BugReports field

Trying a couple of searches gives us a promising result: https://github.com/tidyverse/lubridate/issues?q=is%3Aissue+is%3Aopen+mdy

This is a known bug; it looks like they're thinking about doing something about it, but the only workaround is to create an NA: https://github.com/tidyverse/

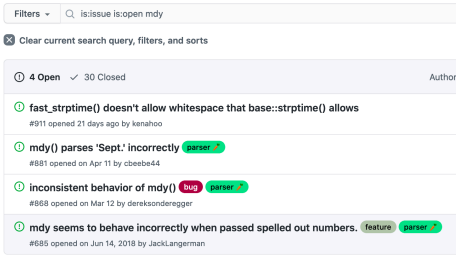


Figure 5.2: Screenshot of `lubridate` issues page, showing a relevant search result

lubridate/issues/685

5.4 Writing a reproducible example: `reprex`

- https://reprex.tidyverse.org/
- https://reprex.tidyverse.org/articles/articles/learn-reprex.html
- https://reprex.tidyverse.org/articles/reprex-dos-and-donts.html

Part II

Exploratory Data Analysis

Chapter 6

Waterfalls, Spirals, and Philosophy of Science

- Waterfall model of development: https://en.wikipedia.org/wiki/Waterfall_model
- Spiral model of development [3]
- "The Scientific Method": https://plato.stanford.edu/entries/scientific-method/#MetMetEndMet

Chapter 7

Exploratory Data Analysis and the Ladder of Models

Part III

Cleaning Data

Part IV

Reproducibility

Part V

Case Study: The COMPAS Dataset

Bibliography

[1] Faizan Ahemad. *Version Control with Jupyter Notebooks*. Oct. 10, 2018. URL: https://towardsdatascience.com/version-control-with-jupyter-notebooks-f096f4d7035a (visited on 07/15/2020).

[2] Julia Angwin et al. *Machine Bias: There's Software Used Across the Country to Predict Future Criminals. And It's Biased Against Blacks.* May 23, 2016. URL: https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing (visited on 05/23/2016).

[3] B. W. Boehm. "A Spiral Model of Software Development and Enhancement". In: *Computer* 21.5 (May 1988), pp. 61–72. ISSN: 1558-0814. DOI: 10.1109/2.59.

[4] John M. Chambers. "Object-Oriented Programming, Functional Programming and R". In: *Statistical Science* 29.2 (May 2014), pp. 167–180. ISSN: 0883-4237. DOI: 10.1214/13-STS452. arXiv: 1409.3531. URL: http://arxiv.org/abs/1409.3531 (visited on 02/28/2020).

[5] Sam Corbett-Davies et al. *A Computer Program Used for Bail and Sentencing Decisions Was Labeled Biased against Blacks. It's Actually Not That Clear.* Oct. 17, 2016. URL: https://www.washingtonpost.com/news/monkey-cage/wp/2016/10/17/can-an-algorithm-be-racist-our-analysis-is-more-cautious-than-propublicas/ (visited on 06/10/2020).

[6] Robert Gentleman and Duncan Temple Lang. "Statistical Analyses and Reproducible Research". In: *Bioconductor Project Working Papers* (May 29, 2004). URL: https://biostats.bepress.com/bioconductor/paper2.

[7] Joel Grus. *I Don't Like Notebooks - Joel Grus - #JupyterCon 2018*. URL: https://docs.google.com/presentation/d/1n2RlMdmv1p25Xy5thJUhkKGvjtV-dkAIsUXP-AL4ffI/edit?usp=embed_facebook (visited on 05/06/2020).

[8] Alex Hanna et al. "Towards a Critical Race Methodology in Algorithmic Fairness". In: (Dec. 7, 2019). DOI: 10.1145/3351095.3372826. arXiv: 1912.03593 [cs]. URL: http://arxiv.org/abs/1912.03593 (visited on 12/17/2019).

[9] Or Hiltch. *Jupyter Notebook Is the Cancer of ML Engineering*. Oct. 17, 2019. URL: https://medium.com/@_orcaman/jupyter-notebook-is-the-cancer-of-ml-engineering-70b98685ee71 (visited on 05/06/2020).

[10] Anna Lauren Hoffmann. "Where Fairness Fails: Data, Algorithms, and the Limits of Antidiscrimination Discourse". In: *Information, Communication & Society* 22.7 (June 7, 2019), pp. 900–915. ISSN: 1369-118X. DOI: 10.1080/1369118X.2019.1573912. URL: https://doi.org/10.1080/1369118X.2019.1573912 (visited on 05/13/2019).

[11] Marianne Huebner, Werner Vach, and Saskia le Cessie. "A Systematic Approach to Initial Data Analysis Is Good Research Practice". In: *The Journal of Thoracic and Cardiovascular Surgery* 151.1 (Jan. 1, 2016), pp. 25–27. ISSN: 0022-5223. DOI: 10.1016/j.jtcvs.2015.09.085. URL: http://www.sciencedirect.com/science/article/pii/S0022522315017948 (visited on 05/12/2020).

[12] Os Keyes. *Rbitrary Standards*. Mar. 12, 2016. URL: https://ironholds.org/projects/rbitrary/ (visited on 03/27/2016).

[13] Jon Kleinberg, Sendhil Mullainathan, and Manish Raghavan. "Inherent Trade-Offs in the Fair Determination of Risk Scores". In: (Nov. 17, 2016). arXiv: 1609.05807 `[cs, stat]`. URL: http://arxiv.org/abs/1609.05807 (visited on 06/10/2020).

[14] Giuseppe Paleologo. *What's the Difference between Machine Learning in Python and R?* Feb. 28, 2018. URL: https://www.quora.com/Whats-the-difference-between-machine-learning-in-Python-and-R/answer/Giuseppe-Paleologo (visited on 03/01/2018).

[15] Jeffrey M. Perkel. "Why Jupyter Is Data Scientists' Computational Notebook of Choice". In: *Nature* 563 (Oct. 30, 2018), p. 145. DOI: 10.1038/d41586-018-07196-1. URL: http://www.nature.com/articles/d41586-018-07196-1 (visited on 10/30/2018).

[16] Emma Pierson et al. "A Large-Scale Analysis of Racial Disparities in Police Stops across the United States". In: *Nature Human Behaviour* (May 4, 2020), pp. 1–10. ISSN: 2397-3374. DOI: 10.1038/s41562-020-0858-1. URL: https://www.nature.com/articles/s41562-020-0858-1 (visited on 05/06/2020).

[17] Andrew D. Selbst et al. *Fairness and Abstraction in Sociotechnical Systems*. SSRN Scholarly Paper ID 3265913. Rochester, NY: Social Science Research Network, Aug. 23, 2018. URL: https://papers.ssrn.com/abstract=3265913 (visited on 11/29/2018).

[18] Gordon Shotwell. *Why I Use R*. Dec. 30, 2019. URL: https://blog.shotwell.ca/posts/why_i_use_r/ (visited on 06/02/2020).

[19] Tricia Wang. *Why Big Data Needs Thick Data*. Jan. 20, 2016. URL: https://medium.com/ethnography-matters/why-big-data-needs-thick-data-b4b3e75e3d7#.ur5gl84nj (visited on 01/22/2016).

[20] Jonathan Whitmore. *Jupyter Notebook Best Practices for Data Science*. Sept. 10, 2015. URL: https://www.svds.com/jupyter-notebook-best-practices-for-data-science/ (visited on 05/06/2020).

[21] Hadley Wickham. *Advanced R*. CRC Press, Sept. 25, 2014. 474 pp. ISBN: 978-1-4665-8697-0. Google Books: G5PNBQAAQBAJ. URL: https://adv-r.hadley.nz.

[22] Mark D. Wilkinson et al. "The FAIR Guiding Principles for Scientific Data Management and Stewardship". In: *Scientific Data* 3.1 (1 Mar. 15, 2016), pp. 1–9. ISSN: 2052-4463. DOI: 10.1038/sdata.2016.18. URL: https://www.nature.com/articles/sdata201618 (visited on 05/09/2020).

[23] Greg Wilson et al. "Good Enough Practices in Scientific Computing". In: *PLOS Computational Biology* 13.6 (June 22, 2017), e1005510. ISSN: 1553-7358. DOI: 10.1371/journal.pcbi.1005510. URL: https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1005510 (visited on 10/07/2018).

[24] Jun Wu. *Python vs. R — Choosing the Best Programming Language for Data Science*. June 25, 2019. URL: https://towardsdatascience.com/python-vs-r-choosing-the-best-programming-languages-for-data-science-b1327f01f6bf (visited on 07/15/2020).